

ANÁLISIS Y DISEÑO DE ALGORITMOS I

PRÁCTICO: TÉCNICAS DE DISEÑO DE ALGORITMOS: DIVIDE Y CONQUISTA

- 1) Implemente en C++ los métodos de ordenamiento *mergesort* y *quicksort* de forma tal que, dado un arreglo de elementos comparables, ordenen sus elementos ascendentemente.
- 2) Sea T un arreglo ordenado de n enteros diferentes (puede incluir números negativos):
 - a) Implemente un algoritmo en C++ que encuentre el índice i tal que $0 \leq i < n$ y $T[i] = i$, siempre y cuando exista tal índice. La complejidad temporal de la solución propuesta debe pertenecer a $O(\log n)$.
 - b) Explicar por qué una búsqueda binaria es esencial en grandes volúmenes de datos (nube, redes sociales, etc.)
- 3) Se tiene un arreglo T de n enteros distintos, ordenado en forma creciente. Dado un rango definido por dos valores enteros x e y , con la condición de que $x \leq y$, se requiere implementar un algoritmo en C++ que determine cuántos elementos del arreglo T se encuentran comprendidos entre x e y , inclusive. La complejidad temporal de su algoritmo debe pertenecer a $O(\log n)$.
- 4) En ciertas aplicaciones, puede ser necesario combinar información proveniente de distintas fuentes sin alterar su estructura original. Supongamos que se tienen dos vectores X e Y , cada uno con n enteros ordenados en forma creciente. Se desea calcular la mediana del conjunto total de $2n$ elementos sin necesidad de mezclar ni ordenar explícitamente los vectores.
Implemente un algoritmo en C++ que resuelva el problema en un tiempo $O(\log n)$.
- 5) En aplicaciones de monitoreo (como salud, clima o rendimiento físico), es común analizar secuencias de datos para identificar comportamientos que alcanzan un pico y luego decrecen, lo que se conoce como una secuencia unimodal. Formalmente, una secuencia $A = (a_1, a_2, \dots, a_n)$ de longitud $n \geq 3$ se llama *unimodal* si sus elementos primero aumentan y luego disminuyen, es decir existe un índice p con $1 \leq p \leq n$ tal que $a_1 < a_2 < \dots < a_p$ y $a_p > a_{p+1} > \dots > a_n$. Implemente un algoritmo en C++ que dado un arreglo unidimensional que almacena una secuencia unimodal encuentre el índice p . La complejidad temporal del algoritmo debe pertenecer a $O(\log n)$.
- 6) Supongamos que se tiene un arreglo que contiene datos recolectados por dispositivos *wearables*, como pulseras inteligentes o relojes deportivos. Este arreglo almacena mediciones de actividad física registradas durante un cierto período de tiempo. Los datos pueden corresponder a un único usuario o a múltiples usuarios. Se desea determinar el *valor mínimo* y el *valor máximo* de actividad física registrados en el conjunto de datos, con el objetivo de detectar posibles patrones de comportamiento. Por ejemplo, identificar posibles extremos en el comportamiento, como días con niveles inusualmente bajos o altos de actividad, lo cual puede ser útil para generar recomendaciones o alertas.

ANÁLISIS Y DISEÑO DE ALGORITMOS I

PRÁCTICO: TÉCNICAS DE DISEÑO DE ALGORITMOS: DIVIDE Y CONQUISTA

- a) Resolver el problema implementando un algoritmo iterativo en lenguaje C++.
 - b) Resolver el problema utilizando la técnica de divide y conquista, también en C++.
 - c) Calcular la complejidad temporal de cada uno de los algoritmos implementados en los puntos a) y b).
 - d) Realizar un análisis comparativo de eficiencia.
- 7) Tu plataforma de e-commerce administra un arreglo ordenado S que contiene n cupones, representados por valores monetarios enteros y distintos. Dado un valor entero x (que representa un precio objetivo de compra), se desea determinar si existe al menos un par de cupones en el arreglo cuya suma sea exactamente igual a x
- a) Implementar un algoritmo en C++ que permita determinar si existe dicho par de cupones.
 - b) Calcule la complejidad temporal.
- 8) Se dispone de un arreglo de enteros que representa las variaciones registradas por sensores de pulso o ECG (electrocardiograma) a lo largo de un período de tiempo. Los valores positivos indican aumentos en la señal, y los negativos, descensos. Se desea analizar la señal para identificar el intervalo más significativo en cuanto a variación, es decir, el subarreglo contiguo cuya suma sea máxima.
- a) Implementar un algoritmo en C++ que identifique el subarreglo contiguo con la mayor suma posible. Este intervalo representa el período donde se observa la mayor variación acumulada en la señal medida.
 - b) Determinar la complejidad temporal y analizar qué ocurre si todos los valores del arreglo fueran positivos. ¿Es necesario aplicar la misma técnica en ese caso o existe una solución más simple?
- 9) Supongamos que tenemos 2 algoritmos alternativos para resolver un problema de tamaño n :
- a) El algoritmo A resuelve el problema dividiéndolo en dos subproblemas de tamaño $n/2$, resuelve recursivamente cada subproblema y combina la solución en tiempo lineal.
 - b) El algoritmo B resuelve el problema resolviendo dos subproblemas de tamaño $n-1$ y combinando la solución en tiempo constante.
- ¿Cuál es el tiempo de ejecución de cada uno de esos algoritmos (en notación Big-O) y cuál escogería para resolver el problema?